Informe 4

Autores:

Eric Guacache C.I: 31.788.759
Renny Andrade C.I: 29.727.828

Índice

1. Ciclo de una Interrupción de Hardware **2**

2. Diferencias entre Gestión de E/S por Sondeo y por Interrupciones **3**

3. Ventajas del uso de interrupciones en términos de uso del procesador **4**

4. Registros especiales en MIPS32 para gestionar interrupciones **4**

5. Importancia de guardar el contexto al entrar en una rutina de servicio **4**

6. Excepciones en un sistema MIPS32 **5**
6.1. Situaciones que generan excepciones 5
6.2. Etapas del pipeline que pueden provocar una excepción 5

7. Estrategias de tratamiento de excepciones e interrupciones **6**
7.1. Diferencias entre Interrupciones y Excepciones 6
7.2. Estrategias de Tratamiento de Excepciones en MIPS32 6

8. Habilitación de interrupciones en dispositivos y procesador **7**
8.1. Cómo se habilitan las interrupciones 7
8.2. ¿Qué pasaría si las interrupciones se habilitan solo en los dispositivos? . 7

9. Procesamiento de interrupciones **8**
9.1. Proceso de una Interrupción de Reloj 8
9.2. Importancia de guardar el contexto 9

10. Interrupciones de reloj y control de ejecución **9**
10.1. Uso de Interrupciones de Reloj para Control de Ejecución 9
10.2. Acción del Software si el Programa Finaliza Temprano 9

11. Análisis y Discusión de los Resultados **10**

1. Ciclo de una Interrupción de Hardware

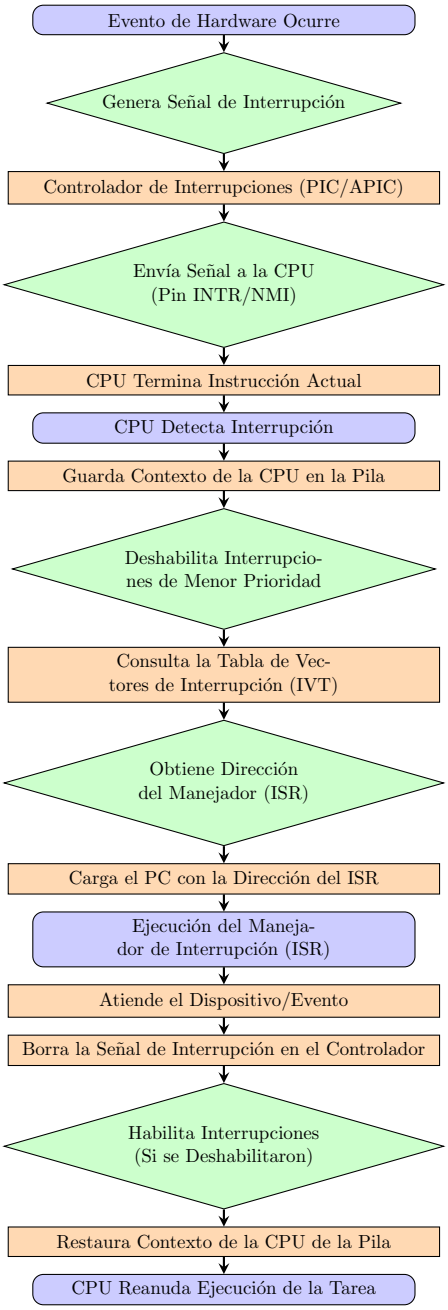


Figura 1: Diagrama de ciclo de interrupción de hardware

El ciclo de una interrupción de hardware es una secuencia de pasos que la CPU y el sistema operativo siguen para atender una solicitud de un dispositivo de hardware.

1. **Evento de Hardware Ocurre:** Un dispositivo de hardware necesita atención.
2. **Genera Señal de Interrupción:** El dispositivo envía una señal al controlador de interrupciones.
3. **Controlador de Interrupciones (PIC/APIC):** Recibe la señal, la prioriza y la envía a la CPU.

4. **Envía Señal a la CPU (Pin INTR/NMI):** La señal llega a un pin específico de la CPU.
5. **CPU Termina Instrucción Actual:** La CPU finaliza la instrucción en curso.
6. **CPU Detecta Interrupción:** La CPU reconoce la señal de interrupción.
7. **Guarda Contexto de la CPU en la Pila:** La CPU guarda su estado actual (PC, registros, PSW) en la pila para poder reanudar la tarea original.
8. **Deshabilita Interrupciones de Menor Prioridad (Opcional):** Se pueden deshabilitar temporalmente interrupciones menos críticas.
9. **Consulta la Tabla de Vectores de Interrupción (IVT):** La CPU usa el número de interrupción para encontrar la dirección del manejador de interrupción (ISR) en la IVT/IDT.
10. **Obtiene la Dirección del Manejador de Interrupción (ISR):** Se recupera la dirección del ISR.
11. **Carga el PC con la Dirección del ISR:** El control de ejecución se transfiere al ISR.
12. **Ejecución del Manejador de Interrupción (ISR):** El ISR se ejecuta para atender la solicitud del dispositivo.
13. **Atiende el Dispositivo/Evento:** El ISR resuelve la causa de la interrupción.
14. **Borra la Señal de Interrupción en el Controlador:** El ISR notifica al controlador que la interrupción ha sido atendida.
15. **Habilita Interrupciones (Si se Deshabilitaron):** Las interrupciones previamente deshabilitadas se vuelven a habilitar.
16. **Restaura Contexto de la CPU de la Pila:** La CPU recupera su estado guardado de la pila.
17. **CPU Reanuda Ejecución de la Tarea Pre-Interrupción:** La CPU continúa ejecutando la tarea que fue interrumpida.

2. Diferencias entre Gestión de E/S por Sondeo y por Interrupciones

La gestión de E/S por **sondeo (polling)** implica que la CPU pregunta repetidamente a un dispositivo si necesita servicio, actuando como un ciclo continuo de verificación activa. Este mecanismo es más simple de implementar, pero a menudo resulta menos eficiente, ya que la CPU puede desperdiciar ciclos de reloj preguntando a dispositivos inactivos, lo que lleva a un alto uso de la CPU y una menor productividad. La latencia puede ser variable y potencialmente alta, aunque en sistemas muy simples o con E/S extremadamente predecible y frecuente, el sondeo puede tener una latencia

baja. Se utiliza en sistemas básicos o cuando las operaciones de E/S son muy frecuentes y predecibles, e incluso en algunos casos de dispositivos de ultra-baja latencia para evitar la sobrecarga de interrupciones.

Por otro lado, la gestión de E/S por **interrupciones** ocurre cuando el dispositivo de E/S señala a la CPU solo cuando requiere atención o ha terminado una tarea, notificándola pasivamente. Este método es más eficiente en la mayoría de los escenarios, ya que la CPU puede realizar otras tareas mientras espera, reduciendo su tiempo de inactividad. Aunque es más complejo de implementar, requiriendo rutinas de servicio de interrupción (ISR) y gestión de cambio de contexto, ofrece una latencia más baja y un tiempo de respuesta más rápido. Es ideal para la mayoría de los sistemas operativos modernos y aplicaciones donde la capacidad de respuesta es crítica, como en sistemas en tiempo real o cuando las operaciones de E/S son impredecibles.

3. Ventajas del uso de interrupciones en términos de uso del procesador

El uso de interrupciones ofrece ventajas significativas en términos de uso del procesador, ya que permite que la CPU realice otras tareas mientras espera la finalización de las operaciones de E/S, reduciendo así su tiempo de inactividad. Esto resulta en una mayor eficiencia y capacidad de respuesta del sistema, siendo ideal para entornos donde la inmediatez es crucial.

4. Registros especiales en MIPS32 para gestionar interrupciones

En la arquitectura MIPS32, la gestión de interrupciones se realiza principalmente a través de registros especiales del **coprocesador 0 (CP0)**. Los más importantes son:

- **Status**: Controla la habilitación global de interrupciones y el estado del procesador.
- **Cause**: Indica la fuente de la interrupción.
- **EPC**: Almacena la dirección de retorno del programa interrumpido.

Estos registros permiten identificar y manejar las interrupciones para que el procesador pueda reanudar su tarea original.

5. Importancia de guardar el contexto al entrar en una rutina de servicio

Es necesario guardar el contexto (registros) al entrar en una rutina de servicio de interrupción para preservar el estado del programa que fue interrumpido. De esta manera, una vez que la interrupción ha sido procesada, el programa original puede reanudar su ejecución desde el punto exacto donde fue interrumpido, sin perder información ni causar errores.

6. Excepciones en un sistema MIPS32

6.1. Situaciones que generan excepciones

En un sistema MIPS32, las excepciones pueden generarse por diversas situaciones, interrumpiendo el flujo normal de ejecución del programa para que el sistema operativo pueda manejarlas. Algunas de las situaciones comunes incluyen:

- **Desbordamiento Aritmético (Overflow):** Ocurre cuando el resultado de una operación aritmética excede la capacidad del registro de destino.
- **Fallo de Dirección (Address Error):** Se produce al intentar acceder a una dirección de memoria no alineada o inválida (por ejemplo, una dirección de instrucción o de dato que no cumple con los requisitos de alineación del hardware).
- **Instrucción Ilegal/Desconocida:** Sucede cuando el procesador intenta ejecutar una instrucción cuyo código de operación no es reconocido o es inválido.
- **Llamada al Sistema (Syscall):** Es una excepción generada intencionalmente por software para solicitar servicios al sistema operativo (por ejemplo, operaciones de E/S, gestión de memoria).
- **Fallo de Página / TLB Miss:** Ocurre cuando la Unidad de Gestión de Memoria (MMU) no puede traducir una dirección virtual a una física porque la entrada correspondiente no se encuentra en la TLB (Translation Lookaside Buffer) o la página no está presente en memoria física.
- **Interrupción de Hardware:** Causada por un evento externo al procesador, como la finalización de una operación de E/S por un periférico, una señal de temporizador, o una entrada de teclado.

6.2. Etapas del pipeline que pueden provocar una excepción

En un pipeline MIPS típico de 5 etapas (Fetch, Decode, Execute, Memory, Write-back), las excepciones pueden originarse en distintas fases:

- **IF (Instruction Fetch - Búsqueda de Instrucción):**
  - **Fallo de Dirección:** Si la dirección de la instrucción a buscar es inválida o no está alineada.
  - **Fallo de Página/TLB Miss:** Si la traducción de la dirección virtual de la instrucción falla en la TLB.
- **ID (Instruction Decode - Decodificación de Instrucción):**
  - **Instrucción Ilegal/Desconocida:** Si el opcode de la instrucción decodificada no es válido.
  - **Coprocesador No Usable:** Si la instrucción intenta usar un coprocesador que no está habilitado o presente.
- **EX (Execute - Ejecución):**

    - **Desbordamiento Aritmético:** Para instrucciones aritméticas que producen un resultado que excede el rango representable.

- **MEM (Memory Access - Acceso a Memoria):**
    - **Fallo de Dirección:** Para instrucciones de carga o almacenamiento si la dirección de los datos es inválida o no alineada.
    - **Fallo de Página/TLB Miss:** Si la traducción de la dirección virtual de los datos falla en la TLB.
- **WB (Write-Back - Escritura de Resultado):**
    - Aunque las excepciones se detectan en etapas anteriores, esta etapa es donde el resultado de una operación que causó una excepción (como un desbordamiento) podría intentarse escribir, pero la excepción ya habría sido señalada. La etapa de escritura en sí misma no suele ser la causa directa de una nueva excepción, sino el punto donde se haría evidente el impacto de una excepción anterior si no se manejara.

7. Estrategias de tratamiento de excepciones e interrupciones

7.1. Diferencias entre Interrupciones y Excepciones

Las **interrupciones** son eventos asíncronos generados externamente al procesador (ej., un dispositivo de E/S que termina una operación o un temporizador). Las **excepciones** son eventos síncronos generados internamente por el propio procesador al ejecutar una instrucción (ej., desbordamiento aritmético, instrucción ilegal, fallo de página o una llamada al sistema). Ambas desvían el flujo normal de ejecución para ser manejadas por el sistema operativo.

7.2. Estrategias de Tratamiento de Excepciones en MIPS32

En MIPS32, existen principalmente dos estrategias para tratar excepciones:

- **Manejo Único (o No Vectorizado):** Todas las excepciones redirigen la ejecución a una única dirección de entrada fija para la rutina de servicio de excepciones. Una vez allí, el software lee el registro `Cause` para determinar el tipo específico de excepción y saltar a la rutina de manejo adecuada.
- **Manejo Vectorizado:** Cada tipo de excepción (o grupo de tipos) redirige la ejecución a una dirección de entrada única y predefinida para ese tipo específico. Esto permite un manejo más rápido al evitar la necesidad de que el software identifique la causa inicial.

En ambos casos, la ejecución se redirige automáticamente por hardware. Antes de saltar a la rutina de servicio, el hardware guarda la dirección de la instrucción que causó la excepción (o la siguiente instrucción, para excepciones precisas) en el registro **EPC (Exception Program Counter)**. La función del registro EPC es crucial: permite que, una vez que la excepción ha sido manejada, el programa original pueda

reanudar su ejecución desde el punto correcto, garantizando la continuidad del proceso interrumpido.

8. Habilitación de interrupciones en dispositivos y procesador

8.1. Cómo se habilitan las interrupciones

- **Teclado:** La habilitación de interrupciones en el teclado se realiza configurando el **controlador de teclado** (hardware). Esto implica escribir un valor específico en uno de sus registros de control para que genere una señal de interrupción al procesador cada vez que se presiona o suelta una tecla.
- **Pantalla:** Para la pantalla (o su **controlador gráfico**), las interrupciones se habilitan de manera similar. Se modifican los registros de control del controlador gráfico para que genere interrupciones en eventos específicos, como la finalización de un ciclo de refresco vertical (VBLANK) o la conclusión de una operación gráfica.
- **Procesador (MIPS32):** En el procesador MIPS32, las interrupciones se habilitan a través del registro **Status** (parte del coprocesador 0 - CP0). Los bits clave a modificar son:
  - **IE (Interrupt Enable) - Bit 0:** Este bit controla la habilitación/deshabilitación global de todas las interrupciones. Debe establecerse a '1' para permitir que el procesador responda a las interrupciones.
  - **IM (Interrupt Mask) - Bits 8-15:** Estos bits actúan como una máscara para interrupciones individuales o grupos de interrupciones. Cada bit corresponde a una línea de interrupción específica; al establecer un bit a '1', se habilita la interrupción correspondiente.
  - **EXL (Exception Level) - Bit 1:** Cuando este bit está a '1' (indicando que el procesador está manejando una excepción), las interrupciones adicionales suelen ser deshabilitadas para evitar interrupciones anidadas no deseadas. Para habilitar interrupciones, este bit debe estar a '0' o gestionarse adecuadamente en la rutina de servicio.

8.2. ¿Qué pasaría si las interrupciones se habilitan solo en los dispositivos?

Si las interrupciones se habilitan en los dispositivos (teclado, pantalla, etc.), pero no en el procesador (es decir, el bit IE del registro Status del MIPS32 está a '0'), los dispositivos generarán las señales de interrupción cuando ocurran los eventos. Sin embargo, el procesador **ignorará estas señales**. Las interrupciones podrían quedar pendientes en el registro `Cause` del CP0, pero la CPU no desviará su flujo de ejecución hacia la rutina de servicio de interrupción. En consecuencia, el sistema no responderá a los eventos de los dispositivos, lo que se traduciría en una falta de interacción o funcionalidad del hardware.

9. Procesamiento de interrupciones

9.1. Proceso de una Interrupción de Reloj

Cuando se produce una interrupción de reloj en un sistema MIPS32, la secuencia de eventos es la siguiente:

- **Evento y Detección (Hardware):** El temporizador (hardware) genera una señal de interrupción. Si las interrupciones están habilitadas globalmente en el procesador (bit IE en el registro Status) y la interrupción específica del reloj está enmascarada (bit IM correspondiente en Status), el hardware del procesador detecta la señal.
- **Guardado de Contexto Mínimo (Hardware):**
  - El hardware guarda la dirección de la instrucción actual (o la siguiente, dependiendo de la precisión de la excepción) en el registro **EPC (Exception Program Counter)**.
  - El hardware actualiza el registro **Status** para deshabilitar temporalmente las interrupciones (estableciendo el bit EXL a '1' y/o el bit IE a '0') y cambiar al modo privilegiado (kernel).
  - El registro **Cause** se actualiza para indicar que la interrupción de reloj es la causa.
- **Redirección de Ejecución (Hardware):** El hardware redirige el flujo de ejecución a la dirección de la rutina de servicio de interrupción (ISR) predefinida por el sistema (generalmente un vector de interrupción).
- **Guardado de Contexto Adicional (Software - Rutina de Servicio):** Una vez dentro de la ISR, el software (sistema operativo o la rutina de servicio) es responsable de guardar los **registros de propósito general** (`t0-t9`, `s0-s7`, etc.) que la rutina vaya a utilizar en la pila del kernel. Esto es crucial para evitar corromper los valores que el programa interrumpido estaba utilizando.
- **Procesamiento de la Interrupción (Software):** La ISR ejecuta el código necesario para manejar el evento del reloj (ej., actualizar el contador de tiempo del sistema, programar la próxima interrupción del temporizador, o despertar procesos).
- **Restauración de Contexto (Software - Rutina de Servicio):** Antes de finalizar, la ISR restaura los registros de propósito general que guardó de la pila.
- **Retorno de Interrupción (Hardware/Software):** La ISR ejecuta la instrucción `ERET` (Exception Return). Esta instrucción es crucial:
  - Restaura el registro **Status** a su estado anterior (restableciendo EXL y IE, volviendo al modo de usuario si aplica).
  - Carga el valor de **EPC** en el Contador de Programa (PC), permitiendo que el programa interrumpido reanude su ejecución.

9.2. Importancia de guardar el contexto

Es fundamental guardar el contexto (registros generales, EPC, Status) al entrar en una rutina de servicio de interrupción para asegurar la **transparencia y la correcta reanudación** del programa interrumpido. Si no se guardan, la ISR podría sobrescribir valores críticos en los registros del procesador, lo que llevaría a que el programa original se ejecute incorrectamente, produzca resultados erróneos o incluso falle. Guardar el contexto garantiza que, al retornar de la interrupción, el estado del procesador sea idéntico al que tenía justo antes de ser interrumpido, permitiendo una ejecución fluida y sin interrupciones perceptibles para el programa afectado.

10. Interrupciones de reloj y control de ejecución

10.1. Uso de Interrupciones de Reloj para Control de Ejecución

Las interrupciones de reloj son fundamentales para la gestión de procesos en un sistema operativo, permitiendo:

- **Evitar bucles infinitos:** El sistema operativo configura un temporizador para generar una interrupción de reloj periódicamente. Si un programa entra en un bucle infinito, la interrupción de reloj lo detendrá después de un cierto "quantum"de tiempo. La rutina de servicio de interrupción (ISR) del reloj puede entonces tomar el control, detectar que el programa ha excedido su tiempo asignado y, si es necesario, terminarlo o asignarle un nuevo turno de ejecución.
- **Finalizar programas que superan un tiempo máximo de ejecución:** Similar al punto anterior, el sistema operativo puede asignar un tiempo máximo de ejecución a un programa. Si el programa no finaliza antes de que se genere la interrupción de reloj correspondiente a ese límite de tiempo, la ISR del reloj puede identificar al programa que ha excedido su cuota y forzar su terminación, liberando así los recursos del sistema.

10.2. Acción del Software si el Programa Finaliza Temprano

Si un programa finaliza su ejecución antes de que ocurra la interrupción de reloj programada para su "quantum.º tiempo máximo, el software (el sistema operativo) debe:

- **Deshabilitar la interrupción de reloj para ese proceso:** Una vez que el programa termina (por ejemplo, al ejecutar una llamada al sistema para salir), el sistema operativo debe deshabilitar o reconfigurar la interrupción de reloj que estaba asociada específicamente a ese programa. Esto evita que el procesador sea interrumpido innecesariamente por un temporizador que ya no es relevante para un proceso en ejecución.
- **Liberar recursos:** El sistema operativo procederá a liberar todos los recursos asignados a ese programa, incluyendo la entrada del temporizador si era un temporizador dedicado al proceso.

11. Análisis y Discusión de los Resultados

El análisis de las preguntas planteadas sobre la gestión de E/S, interrupciones y excepciones en la arquitectura MIPS32 revela la importancia crítica de estos mecanismos para el funcionamiento eficiente y robusto de cualquier sistema operativo.

En primer lugar, la comparación entre la gestión de E/S por **sondeo** y por **interrupciones** subraya una diferencia fundamental en la eficiencia del procesador. Mientras que el sondeo consume ciclos de CPU al verificar constantemente el estado de los dispositivos, las interrupciones permiten que el procesador se dedique a otras tareas hasta que un dispositivo lo notifica activamente. Esta capacidad de respuesta asíncrona es una ventaja clave de las interrupciones, liberando recursos del procesador y mejorando significativamente el rendimiento general del sistema.

La distinción entre **interrupciones** (eventos asíncronos externos) y **excepciones** (eventos síncronos internos generados por la CPU) es crucial para comprender cómo el procesador maneja situaciones inesperadas o solicitudes de servicio. En MIPS32, registros especiales del **coprocesador 0 (CP0)**, como `Status`, `Cause` y `EPC (Exception Program Counter)`, son fundamentales para la gestión de estos eventos. El `Status` controla la habilitación y el estado del procesador, `Cause` identifica la fuente del evento, y `EPC` es vital para guardar la dirección de retorno, permitiendo que el programa interrumpido reanude su ejecución de forma transparente. La necesidad de **guardar el contexto** (registros generales, EPC, Status) al entrar en una rutina de servicio es, por tanto, indispensable para preservar la integridad del programa original y garantizar una reanudación sin errores.

Finalmente, las **interrupciones de reloj** emergen como una herramienta poderosa para el control de la ejecución de programas. Permiten al sistema operativo evitar que los programas queden en bucles infinitos o excedan su tiempo de ejecución asignado, al forzar un cambio de contexto periódico. Esto es esencial para la multitarea y la equidad en el uso del procesador. Si un programa finaliza antes de la interrupción de reloj esperada, el software debe deshabilitar proactivamente esa interrupción y liberar los recursos asociados para evitar interrupciones innecesarias y optimizar el uso del sistema. En conjunto, estos mecanismos demuestran cómo el hardware y el software colaboran para crear un entorno de ejecución fiable y eficiente.